Online Examination Platform on Kubernetes

Project Overview

This project demonstrates the deployment of a microservices-based Online Examination Platform on a self-managed Kubernetes cluster running on AWS EC2 instances.

The application consists of multiple services:

- Frontend Service (React + Vite)
- Authentication Service (Django)
- Exam Service
- Result Service
- PostgreSQL Database

The infrastructure components include:

- Kubernetes (kubeadm)
- Containerd Runtime
- Calico CNI
- Longhorn Distributed Storage
- NGINX Ingress Controller
- Amazon ECR
- AWS EC2

The goal of this project was not only to deploy an application but also to gain hands-on experience with Kubernetes networking, storage, ingress, image management, troubleshooting, and microservices deployment.

Architecture

```
Internet
    |
NGINX Ingress Controller
    |
+-----------------------------+
| Frontend Service (React)    |
| Auth Service (Django)       |
| Exam Service                |
| Result Service              |
+-----------------------------+
             |
             |
```

```
PostgreSQL
     |
Longhorn
     |
Kubernetes
     |
Calico
     |
AWS EC2 Nodes
```

Infrastructure Setup

Kubernetes Cluster

The Kubernetes cluster was created using kubeadm on AWS EC2 instances.

Cluster Components:

- 1 Control Plane Node
- 2 Worker Nodes
- Container Runtime: Containerd
- CNI Plugin: Calico

Why Calico?

Calico provides networking between Kubernetes Pods.

Responsibilities:

- Pod-to-Pod communication
- Network routing
- Network policy enforcement
- Cross-node pod communication

Without Calico:

- Pods cannot communicate
- Services cannot function
- DNS resolution fails

Persistent Storage with Longhorn

Why Longhorn?

By default, Kubernetes storage is tied to the node where a pod is running.

If a PostgreSQL pod moves to another node:

- Local storage is lost
- Database becomes unavailable

Longhorn solves this problem by providing distributed persistent storage.

Benefits:

- Persistent Volumes
- Dynamic Provisioning
- Data Replication
- High Availability
- CSI Integration

Longhorn Components

Longhorn Manager

Responsible for:

- Volume Management
- Replica Management
- Storage Operations

Longhorn UI

Web interface for:

- Viewing volumes
- Managing storage
- Monitoring health

Longhorn Webhooks

Used for:

- Resource Validation
- Admission Control

- Configuration Checks

Container Registry

Amazon ECR

Docker images were stored in Amazon Elastic Container Registry (ECR).

Images:

- frontend-service
- auth-service
- exam-service
- result-service

Authentication to ECR

Problem

Kubernetes nodes were unable to pull images from ECR.

Error:

```
ImagePullBackOff
```

Containerd error:

```
no basic auth credentials
```

Why This Happened

The cluster was deployed using kubeadm on EC2 and not EKS.

Unlike EKS:

- No automatic ECR integration
- No IAM credential provider configured

Containerd could not authenticate to ECR.

Solution

Created Kubernetes Image Pull Secret.

```
kubectl create secret docker-registry ecr-secret
--docker-server=<ECR_REPOSITORY>
--docker-username=AWS
--docker-password=<TOKEN>
```

Referenced secret in deployments:

```
imagePullSecrets:
- name: ecr-secret
```

Result:

- Kubernetes successfully pulled images from ECR

Database Deployment

PostgreSQL

PostgreSQL was deployed as a Stateful application.

Reasons:

- Persistent Data
- Stable Storage
- Database Reliability

Storage was backed by Longhorn Persistent Volumes.

Auth Service Deployment

The Authentication Service was built using Django.

Startup sequence:

```
Container Start
      |
Wait for PostgreSQL
      |
Run Migrations
      |
Start Django Server
```

Entrypoint Script:

```
python manage.py migrate
python manage.py runserver 0.0.0.0:8000
```

NGINX Ingress Controller

Why Ingress?

Without Ingress:

Each service requires:

- NodePort
- Separate Load Balancer

Ingress provides:

- Central Entry Point
- URL-based Routing
- Simplified Traffic Management

Routes

```
/                 -> frontend-service
/api/auth         -> auth-service
/api/exams        -> exam-service
/api/results      -> result-service
```

Major Troubleshooting

Issue 1: Longhorn Installation Failure

Symptoms:

```
longhorn-manager CrashLoopBackOff
longhorn-ui CrashLoopBackOff
```

Error:

```
admission webhook service is not accessible
```

Investigation

Checked:

- Longhorn Pods
- Webhook Logs
- Services
- Endpoints

Observed:

```
CoreDNS lookups failing
```

Example:

```
nslookup kubernetes.default.svc.cluster.local
```

Failed.

Issue 2: DNS Resolution Failure

Symptoms:

```
nslookup kubernetes.default.svc
```

Failed.

Initially appeared to be a DNS issue.

Deep Investigation

Verified:

- CoreDNS Pods Running
- kube-dns Service Exists
- Endpoints Available

However:

```
ping 10.96.0.10
```

Failed.

Even direct pod communication failed.

Root Cause Discovery

The actual problem was not DNS.

The actual issue:

```
Cross-node Pod Communication Failure
```

Pods located on different nodes could not communicate.

This caused:

- DNS failures
- Longhorn failures
- Service communication failures

Root Cause

AWS Security Group was blocking Calico networking traffic.

Calico uses:

- IP-in-IP encapsulation
- Cross-node routing

Required traffic was not allowed.

As a result:

```
Pod A
  |
  X
  |
Pod B
```

Communication failed.

Resolution

Updated Security Group rules to allow required cluster communication.

After fixing Security Group:

- Pod communication restored
- DNS working
- Longhorn installed successfully
- Services reachable

Verification:

```
nslookup kubernetes.default.svc
```

Successful.

Issue 3: Django Migration Failure

Error:

```
django.db.utils.ProgrammingError:
relation "users" does not exist
```

Cause

Custom User model:

```
AUTH_USER_MODEL = 'users.User'
```

Migration dependency issue.

Database schema was not properly initialized.

Resolution

Generated and applied migrations correctly.

```
python manage.py makemigrations
python manage.py migrate
```

After migration:

- Users table created
- Application started successfully

Concepts Learned

Kubernetes

- Pods
- Deployments
- Services
- StatefulSets

- Secrets
- ConfigMaps
- Ingress
- Persistent Volumes
- Storage Classes

Networking

- ClusterIP
- Service Discovery
- CoreDNS
- Calico
- Cross-node Routing
- Security Groups
- Kubernetes DNS

Storage

- Longhorn
- CSI Drivers
- Persistent Volumes
- Dynamic Provisioning

Container Runtime

- Docker Image Build
- Amazon ECR
- Containerd
- Image Pull Authentication

Lessons Learned

1. DNS failures are often symptoms rather than root causes.

2. When troubleshooting Kubernetes networking:

3. Verify Pod-to-Pod communication first.

4. Then verify Services.

5. Then verify DNS.

6. Longhorn depends heavily on healthy cluster networking.

7. Self-managed Kubernetes differs significantly from EKS.

8. Image availability in ECR does not guarantee Kubernetes can pull images.

9. Persistent storage is critical for stateful workloads.

10. Understanding the networking path is essential:

```
Client
   |
Ingress
   |
Service
   |
Pod
```

1. Systematic troubleshooting is more important than memorizing commands.

Final Outcome

Successfully deployed a complete microservices-based Online Examination Platform on a self-managed Kubernetes cluster running on AWS EC2.

Implemented:

- Kubernetes
- Calico Networking
- Longhorn Storage
- PostgreSQL
- Django Services
- React Frontend
- NGINX Ingress
- Amazon ECR Integration

The project provided hands-on experience in real-world Kubernetes deployment, networking, storage management, troubleshooting, and microservices operations.